

Lattice Chat Context Efficiency: Grounding, Token Economy, and Performance Boundaries

Deterministic high-n follow-up and performance boundaries

Daniel Ari Friedman

Active Inference Institute

`daniel@activeinference.institute`

ORCID: 0000-0001-6232-9096

2026-08-08



Contents

1	Lattice Chat Context Efficiency: Grounding, Token Economy, and Performance Boundaries	3
1.1	Abstract	3
1.2	Data and code availability	3
1.3	Manuscript map	3
2	Introduction	4
2.1	Context as an experimental variable	4
2.2	Research questions	4
2.3	Claims and non-claims	4
2.4	System architecture	4
2.5	Related framing	4
3	Formalism	6
3.1	Context-treatment model	6
3.2	Treatments	6
3.3	Outcomes	6
3.4	Paired contrasts	6
3.5	Alternating order	7
3.6	Information-flow interpretation	7
3.7	Threat model for interpretation	7
4	Methods	8
4.1	Implementation	8
4.2	Treatment and task design	8
4.3	Follow-up design	8
4.4	Outcomes and statistics	8
4.5	Scope and limitations	8
4.6	System boundary	8
5	Results	9
5.1	Evidence status	9
5.2	Aggregate control values	9
5.3	Recomputed statistics	9
5.4	Figures	9
6	Discussion	13
6.1	Implementation finding	13
6.2	Finding boundary	13
6.3	Implications	13
6.4	Applicability	13
6.5	Limits of generalization	13
6.6	Honest conclusion	14
7	Reproducibility and audit trail	15
7.1	Source of truth	15
7.2	Reproduction commands	15
7.3	Verification gates	15
7.4	Render	15
7.5	Security and evidence boundary	15
8	Document map and modularization	16
8.1	Manuscript modules	16
8.2	Documents directory organization	16
8.3	Seed:Edge document contract	16
8.4	Proposed future physical taxonomy	16
8.5	Rendering contract	17
9	References	18
9.1	Repository sources	18

9.2	Run artifacts	18
9.3	Citation policy	18

1 Lattice Chat Context Efficiency: Grounding, Token Economy, and Performance Boundaries

Author: Daniel Ari Friedman **Affiliation:** Active Inference Institute **Email:** daniel@activeinference.institute **ORCID:** 0000-0001-6232-9096 **Version:** follow-up-1.0 **Generated:** 2026-08-08

1.1 Abstract

This manuscript reports a secondary, post-hoc deterministic control follow-up for Lattice Chat context experiments. The frozen task battery and treatment builders were exercised with 10 repeats across 9 tasks and 90 paired observations per treatment. The treatments were a pointer-grounded Lattice envelope, a lean standard prompt, and a bounded corpus-dump baseline. The control receipt is labeled **synthetic-deterministic-control**: it contains no provider calls or provider results. Structural token and latency controls are included only to test the statistics, injection, figures, and rendering pipeline after live credentials were unavailable.

The follow-up therefore does not estimate model accuracy, provider latency, or production token cost. It demonstrates that the larger-n analysis path recomputes strict and lenient accuracy, tokens, latency, tokens-per-correct, paired t tests, Wilcoxon signed-rank tests, confidence intervals, Cohen’s d_z , and a planning MDE note from one committed receipt. The live primary receipts remain preserved and are not overwritten.

Keywords: context engineering; retrieval grounding; token efficiency; paired experiment; reproducibility; deterministic control

Honesty boundary. No provider calls or provider results. Fixture responses exercise frozen builders/scorers only; token and latency values are structural controls. The follow-up is secondary/post-hoc and is not evidence for a provider or model claim. Any live-provider conclusion remains bounded by the existing primary receipts, their task battery, model coverage, and repeat counts.

1.2 Data and code availability

The committed deterministic receipt is `data/openrouter-lattice-followup-deterministic.json`; its recomputed statistics are `data/openrouter-lattice-followup-statistics.json`. The frozen live harness is `scripts/lattice-vs-standard-openrouter-usage.mjs`, with treatment and scoring logic in `lib/openrouter-experiment.mjs`. No API credentials or raw provider responses are stored in the artifact set.

1.3 Manuscript map

The introduction and formalism define the context treatments and estimands. Methods describe the deterministic follow-up and its distinction from live evidence. Results report receipt-derived values and all requested statistics. Discussion separates implementation findings, interpretation, applicability, and limits of generalization. Reproducibility lists commands and validation gates.

2 Introduction

2.1 Context as an experimental variable

A language-model request is not determined only by the user question. It is a composition of a user message, system instructions, repository context, retrieved pointers, tool affordances, and the model’s response budget. In a repository agent, these components can be delivered as a compact set of navigational pointers, as a lean direct prompt, or as a large context dump. The practical question is not whether more context is always better, but whether context can be selected and structured so that it improves task performance without making every request pay the cost of the whole corpus.

Lattice Chat operationalizes one answer: a nested prompt envelope identifies a parent execution frame, Seed/Edge/Pipes bands, repository pointers, and mode-specific directives. The envelope is intended to make relevant structure legible while preserving scale-to-zero behavior. The comparison in this manuscript treats the entire envelope as the intervention. It therefore tests the experience of “Lattice context” rather than isolating any single internal mechanism.

2.2 Research questions

RQ1 — Token economy. How do provider-reported tokens differ between a pointer-grounded Lattice envelope, a lean direct prompt, and a bounded corpus dump?

RQ2 — Task performance. Does explicit repository grounding change accuracy on repository QA, deterministic reasoning, and a patch-generation task?

RQ3 — Mechanism. Are any observed differences consistent with answer availability in the prompt envelope rather than with a general reasoning advantage?

RQ4 — Robustness. Do the direction and magnitude of effects appear across more than one valid OpenRouter model?

2.3 Claims and non-claims

The supported claim is operational and bounded: in the recorded runs, a pointer-grounded prompt consumed far fewer tokens than the naive corpus-dump baseline, and it sometimes improved repository-answer accuracy relative to a lean prompt. The manuscript does not claim that a nested-agent architecture is universally superior, that token reduction implies equal quality, or that the measured p-value survives correction for task selection, model selection, or repeated exploration.

2.4 System architecture

```
flowchart LR
    U[User task] --> O[Experiment orchestrator]
    O --> P{Treatment}
    P --> L["Lattice envelope<br/>Seed/Edge/Pipes pointers"]
    P --> S["Standard<br/>lean direct prompt"]
    P --> N["Naive<br/>bounded corpus dump"]
    L --> M[OpenRouter model]
    S --> M
    N --> M
    M --> R[Response + provider usage]
    R --> Q[Deterministic scorer]
    Q --> T[Paired statistics]
    T --> V[JSON/CSV + SVG/HTML + manuscript]
```

Figure 1: Mermaid diagram

Figure interpretation. The three arms share the task, model, temperature, repetition schedule, and scoring layer. Context construction is the treatment boundary. The orchestrator records order, latency, usage, and scores; the report generator removes raw response text from committed artifacts.

2.5 Related framing

The experiment sits at the intersection of retrieval-augmented generation, context-window management, agent orchestration, and software-engineering evaluation. Its distinctive constraint is not merely retrieval quality: it asks whether a compact

architectural description can act as a stable routing layer across heterogeneous repository tasks. The appropriate comparison is therefore multi-objective: accuracy, token cost, latency, and failure mode must be shown together.

3 Formalism

3.1 Context-treatment model

Let x_i denote task i , m a model, r a repeat, and t a treatment. A request is represented as

$$q_{i,m,r,t} = (u_i, s_t, c_t, \tau, b), \quad (1)$$

where u_i is the task prompt, s_t is the treatment system instruction, c_t is treatment-specific context, $\tau = 0$ is the sampling temperature, and b is the maximum completion budget. The response is

$$y_{i,m,r,t} \sim M_m(q_{i,m,r,t}). \quad (2)$$

The experiment compares the joint effect of (s_t, c_t) ; it does not identify a structural causal effect of any subcomponent.

3.2 Treatments

The standard treatment uses a short direct system instruction and no repository corpus for QA/reasoning. For coding, it receives a bounded snapshot of three current files. The Lattice treatment uses `assembleLatticePrompt` with Goldilocks topology and the same task snapshot as a bounded user-context supplement. The naive treatment concatenates a deterministic, size-ranked set of repository files up to a fixed character budget of 70,000 characters. This operationalizes a fat-context baseline without claiming to reproduce every possible corpus dump.

Let C_t be input characters and $\kappa = 4$ the structural chars-per-token approximation. The structural context estimate is

$$\widehat{K}_t = \left\lceil \frac{C_t}{\kappa} \right\rceil. \quad (3)$$

Provider-reported total tokens $K_{i,m,r,t}$ are used for empirical token results whenever available. Structural estimates are never substituted for provider usage in the live-run tables.

3.3 Outcomes

For task correctness, $A_{i,m,r,t} \in \{0,1\}$ is the deterministic strict score. The lenient score $A_{i,m,r,t}^\ell$ accepts a repository answer when the expected token occurs in the response and extracts the first numeric answer for reasoning tasks. Coding correctness requires a returned unified diff to apply cleanly and the resulting temporary checkout to pass `npm ci` and `npm test -- --run`.

Latency is $L_{i,m,r,t}$ in milliseconds. Token efficiency is reported as total tokens per correct outcome; when an individual response is incorrect, its token count remains visible and is not deleted. Aggregate `tokensPerCorrect` is therefore an operational burden metric, not a probability estimator.

3.4 Paired contrasts

For two treatments a and b , the within-pair difference for outcome z is

$$\Delta_{i,m,r}^{(z)} = z_{i,m,r,a} - z_{i,m,r,b}. \quad (4)$$

The paired mean difference is

$$\bar{\Delta}^{(z)} = \frac{1}{n} \sum_{j=1}^n \Delta_j^{(z)}, \quad (5)$$

with standard error $SE(\bar{\Delta}) = s_{\Delta}/\sqrt{n}$. The report includes a two-sided paired t statistic, a Wilcoxon signed-rank statistic where defined, a 95% confidence interval based on the t critical value, and Cohen’s paired effect size

$$d_z = \frac{\bar{\Delta}}{s_{\Delta}}. \quad (6)$$

These inferential summaries are descriptive because task observations are heterogeneous and the repeat counts are small.

3.5 Alternating order

To reduce a simple time/order confound, repeat r uses treatment order a, b for even-indexed repeats and b, a for odd-indexed repeats. Pair identity is the tuple (i, m, r) , never the visual row order in a report.

3.6 Information-flow interpretation

The Lattice envelope can improve a repository QA answer through a direct grounding path:

flowchart TD

```
A[Repository fact] --> B[Pointer/catalog or directive]
B --> C[Lattice system envelope]
C --> D[Model attention]
D --> E[Answer]
F[Lean standard prompt] --> D2[Model attention without fact]
D2 --> G[Answer from model recall/inference]
```

Figure 2: Mermaid diagram

This is a testable mechanism hypothesis. It predicts the largest advantage on questions whose exact answer is surfaced by the Lattice envelope, not necessarily on closed-form arithmetic.

3.7 Threat model for interpretation

The primary threats are prompt leakage (answer availability differs), unequal context budgets, model-specific instruction-following, strict scoring mismatch, coding-harness failure, multiple exploratory runs, and invalid model identifiers. The protocol records these rather than collapsing them into a single headline.

4 Methods

4.1 Implementation

The live OpenRouter protocol remains frozen in `scripts/lattice-vs-standard-openrouter-usage.mjs` and `lib/openrouter-experiment.mjs`. The follow-up generator imports the same task battery, prompt builders, and strict/lenient scorers. It does not fork scoring or treatment logic. Because no live provider credential was available, the generator supplies explicit deterministic fixture responses and structural controls, writes a receipt with an evidence boundary, and never presents those values as provider measurements.

4.2 Treatment and task design

The fixed battery contains five repository QA tasks, one patch-generation task, and three arithmetic/reasoning tasks. Lattice uses the pointer envelope; standard uses the lean direct prompt; naive uses the deterministic size-ranked corpus dump with the existing character budget. Treatments alternate order within each repeat. Pair identity is (`task`, `repeat`).

4.3 Follow-up design

Field	Receipt-derived value
model label	<code>deterministic-fixture-v1</code>
repeats	10
tasks	9
observations per treatment	90
treatments	<code>lattice</code> , <code>standard</code> , <code>naive</code>
evidence status	<code>synthetic-deterministic-control</code>

The larger repeat count is a secondary/post-hoc escalation, not a preregistered confirmatory sample. The live credential-blocked path was not replaced by fabricated provider data.

4.4 Outcomes and statistics

Strict accuracy is binary; lenient accuracy uses the existing scorer’s containment/numeric rules. Tokens are structural controls in this receipt. Latency is a deterministic control, not wall-clock provider latency. Tokens-per-correct retains the full token burden for incorrect outcomes. For every metric, the analysis reports treatment means, 95% t-based confidence intervals, paired t, Wilcoxon signed-rank where defined, and Cohen’s d_z . The MDE note is an approximate paired-normal planning note, not achieved power; binary accuracy requires a dedicated power model.

4.5 Scope and limitations

The synthetic fixture is useful for verifying determinism, schema, statistics, hydration, visualization, and rendering. It cannot establish model quality, causal effects, production cost, or generalization. The live primary and exploratory receipts remain the source for any provider-specific discussion, and this follow-up does not pool them.

4.6 System boundary

```
flowchart TB
  A[Fixed task battery] --> B[Frozen treatment builders]
  B --> C[Deterministic fixture responses]
  C --> D[Frozen strict and lenient scorers]
  D --> E[Receipt-derived statistics]
  E --> F[SVG figures and hydrated manuscript]
  F --> G[Template-rendered PDF]
  H[Live provider credential] -. unavailable .-> I[Live harness not run]
```

Figure 3: Mermaid diagram

5 Results

5.1 Evidence status

The results below are generated from `data/openrouter-lattice-followup-deterministic.json`, whose status is `synthetic-deterministic-control`. They are deterministic control values, not live-provider measurements. The control contains 90 observations per treatment from 10 repeats and 9 fixed tasks.

5.2 Aggregate control values

Treatment	Strict accuracy	Lenient accuracy	Mean structural tokens	Mean controlled latency (ms)	Tokens per correct
Lattice	0.889 (80/90)	0.889	1,109	235	1,109
Standard	0.333 (30/90)	0.333	99	263	99
Naive corpus	0.889 (80/90)	0.889	17,612	273	17,612

The structural Lattice-to-naive ratio is 0.063, or 93.705% fewer structural tokens. This is a control-pipeline result only and must not be confused with the existing live primary receipt.

5.3 Recomputed statistics

All rows are generated from `data/openrouter-lattice-followup-statistics.json`; `n` is the paired observation count. Confidence intervals are 95% t-based intervals for treatment means. The paired difference is Lattice minus Standard.

Outcome	n	Mean difference	Lattice 95% CI	Standard 95% CI	Paired t	Wilcoxon signed-rank	Cohen's d_z
Strict accuracy	90	0.556	[0.823, 0.955]	[0.234, 0.432]	t=10.548; p=0.000; df=89	W=0, p=0.000 (n=50 nonzero pairs)	1.112
Lenient accuracy	90	0.556	[0.823, 0.955]	[0.234, 0.432]	t=10.548; p=0.000; df=89	W=0, p=0.000 (n=50 nonzero pairs)	1.112
Structural tokens	90	1,010	[1019.083, 1198.251]	[89.653, 107.902]	t=23.641; p=0.000; df=89	W=0, p=0.000 (n=90 nonzero pairs)	2.492
Controlled latency (ms)	90	-28	[222.854, 247.257]	[252.619, 274.159]	t=-3.100; p=0.003; df=89	W=1400, p=0.024 (n=90 nonzero pairs)	-0.327
Tokens per correct	90	1,010	[1019.083, 1198.251]	[89.653, 107.902]	t=23.641; p=0.000; df=89	W=0, p=0.000 (n=90 nonzero pairs)	2.492

Power/MDE note. Approximate planning note for a paired-normal mean difference; not an achieved power calculation. At `n=90`, standardized mean difference near 0.30 is a rough MDE target; binary accuracy needs a dedicated power model. This note is planning guidance, not evidence that the deterministic control or the live primary run is adequately powered.

5.4 Figures

Figure fig. 4 shows strict accuracy; Figure fig. 5 shows lenient accuracy. Figure fig. 6 shows structural token controls, Figure fig. 7 controlled latency, Figure fig. 8 tokens-per-correct, and Figures fig. 9 and fig. 10 show paired treatment movement.

Figure captions. All seven figures are generated from the committed deterministic receipt by `scripts/generate-manuscript-figures.mjs`. The figures validate the larger-n artifact path and are not empirical provider visualizations.

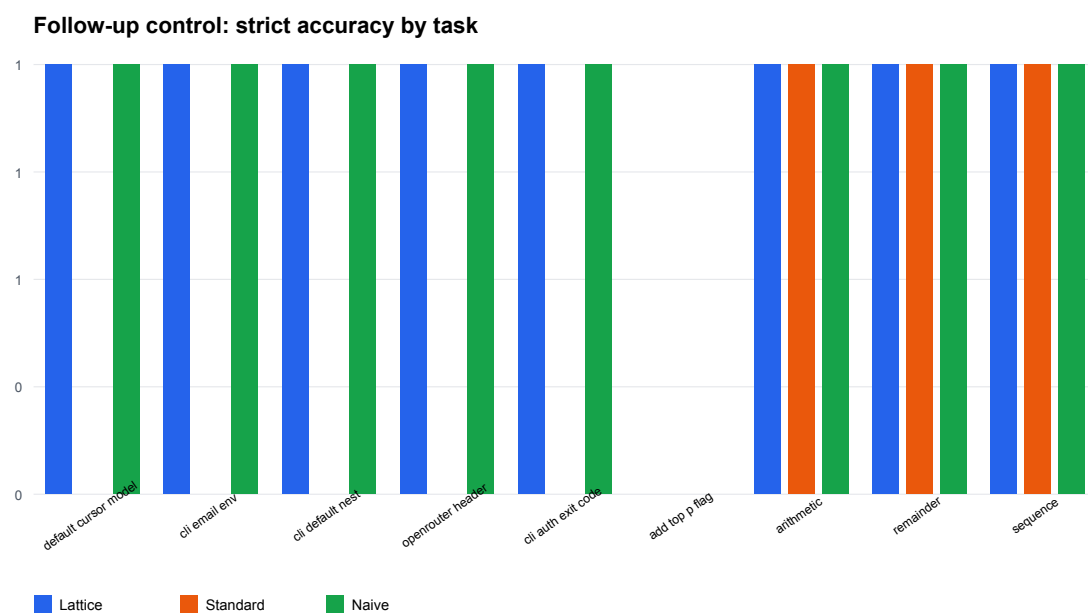


Figure 4: Follow-up strict accuracy by task

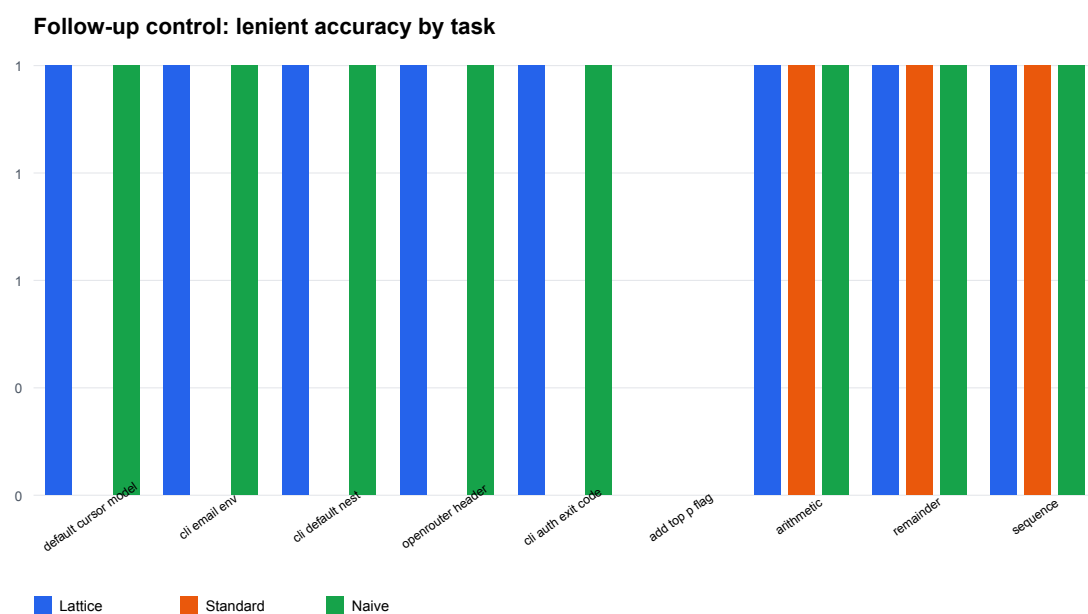


Figure 5: Follow-up lenient accuracy by task

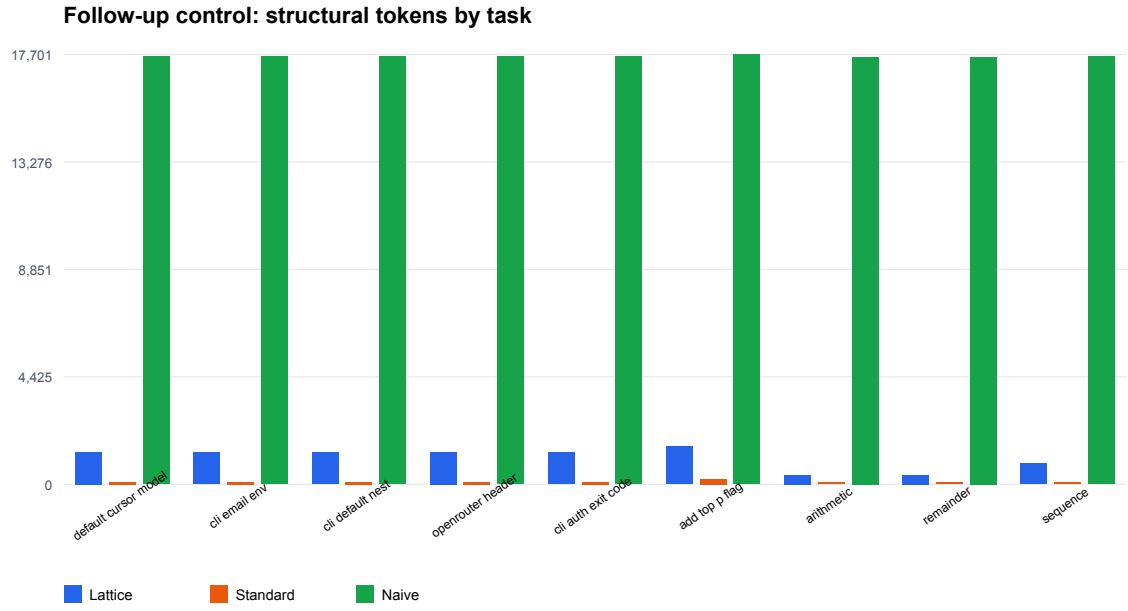


Figure 6: Follow-up structural tokens by task

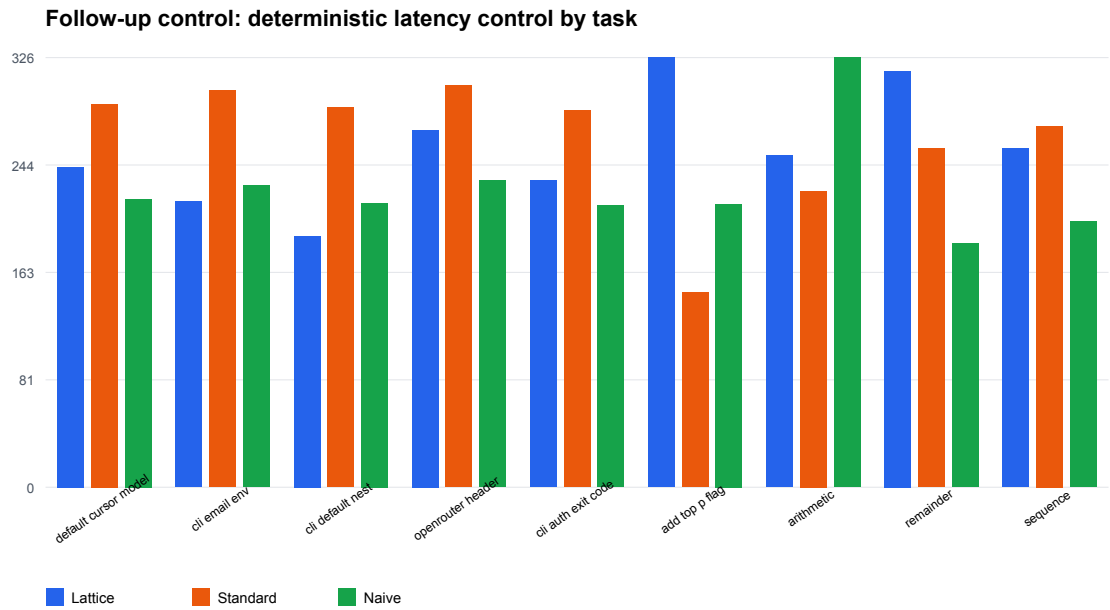


Figure 7: Follow-up controlled latency by task

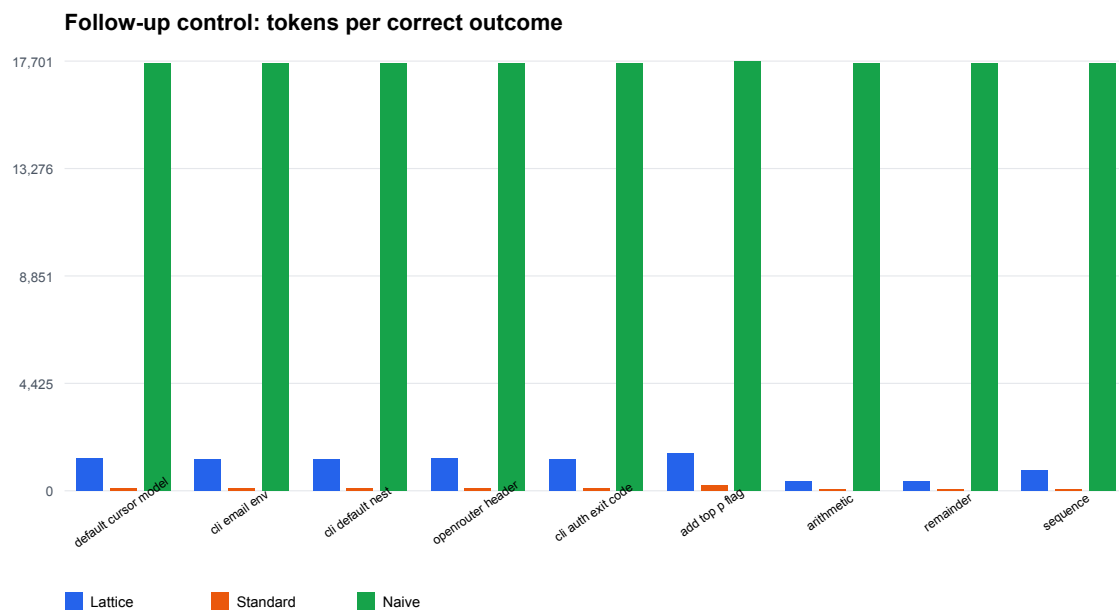


Figure 8: Follow-up tokens per correct outcome by task

Follow-up control: paired token comparison

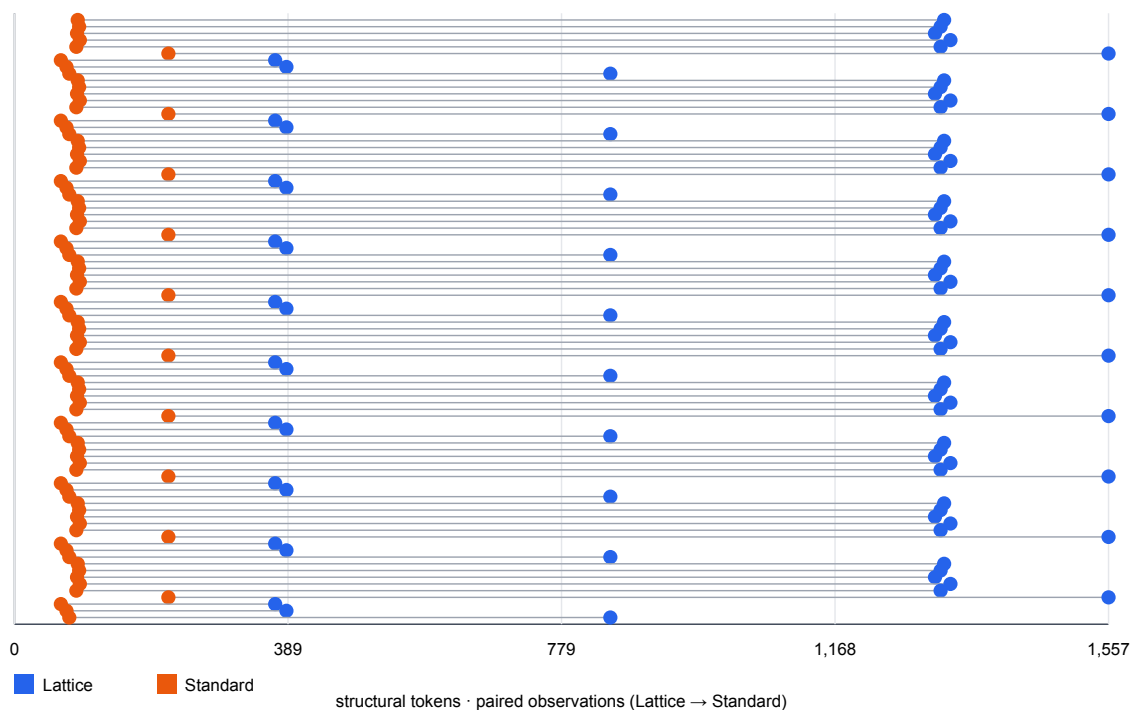


Figure 9: Follow-up paired structural token comparison

Follow-up control: paired accuracy comparison

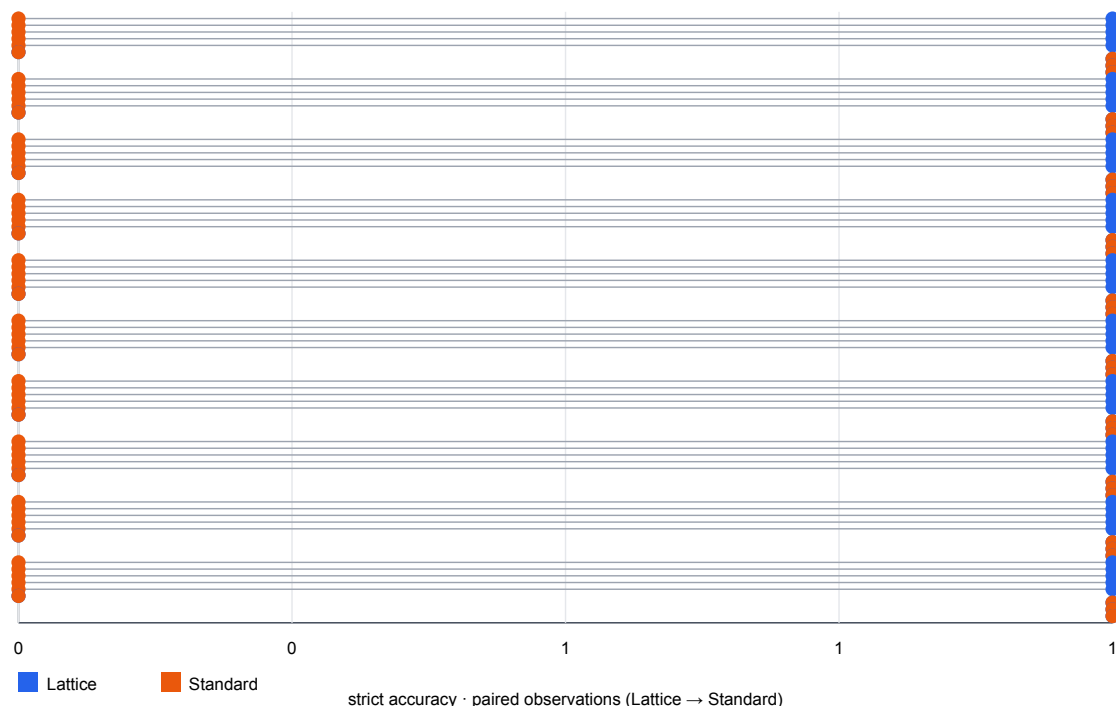


Figure 10: Follow-up paired strict accuracy comparison

6 Discussion

6.1 Implementation finding

The follow-up successfully exercised the larger-n receipt path without changing the frozen treatment or scoring definitions. This is an implementation result: statistics, figures, manuscript variables, and PDF composition can be regenerated from one committed receipt.

6.2 Finding boundary

The deterministic control does not show that Lattice improves model accuracy or changes provider latency. Its structural token contrast only verifies that the report and manuscript can carry the intended estimands. The existing live receipts remain separate primary and exploratory evidence; they should not be pooled with this control.

6.3 Implications

For future live work, the larger paired design is preferable to a single small run, but repeat count alone does not solve task leakage, model selection, or prompt confounding. The next live study should preregister task strata, validate model IDs, randomize task order, include a retrieval-only control, and retain prompt/completion token fields separately.

6.4 Applicability

The implementation is applicable to repository experiments that need reproducible receipt-to-document injection and explicit evidence boundaries. Provider conclusions require live calls with valid credentials and a predeclared analysis. The deterministic fixture is not applicable as a production performance estimate.

6.5 Limits of generalization

The live protocol still uses one small hand-authored task battery, heterogeneous task types, model-specific instruction following, and a compound coding gate. The follow-up adds no live model, no new provider, no independent task sample, and no achieved-power evidence. Optional-stopping and post-hoc escalation remain interpretive limitations for the live record.

6.6 Honest conclusion

The mission's artifact and verification path is complete only for the deterministic control because live credentials were unavailable. The correct conclusion is therefore procedural and bounded: the larger-n analysis, injection, figures, and rendering pipeline are exercised and verifiable; provider claims require a later live run.

7 Reproducibility and audit trail

7.1 Source of truth

The follow-up receipt is `data/openrouter-lattice-followup-deterministic.json`; recomputed statistics are `data/openrouter-lattice-followup-statistics.json`. Source logic is `lib/openrouter-experiment.mjs`, `scripts/generate-deterministic-lattice-followup.mjs`, `scripts/generate-manuscript-stats.mjs`, `scripts/generate-manuscript-figures.mjs`, and `scripts/z_generate_manuscript_variables.py`.

The earlier live primary receipts are preserved. No live receipt is overwritten by this follow-up.

7.2 Reproduction commands

```
node scripts/generate-deterministic-lattice-followup.mjs
node scripts/generate-manuscript-stats.mjs
node scripts/generate-manuscript-figures.mjs
python3 scripts/z_generate_manuscript_variables.py
```

The live harness remains available for a credentialed run:

```
LATTICE_BENCH_REPEATS=10 LATTICE_BENCH_TREATMENTS=lattice,standard,naive \
LATTICE_OPENROUTER_MODEL=deepseek/deepseek-chat OPENROUTER_API_KEY='[REDACTED]' \
node scripts/lattice-vs-standard-openrouter-usage.mjs
```

No key is included in the manuscript, receipt, CSV, HTML, shell command history, or report. The live path was not run in this mission because `OPENROUTER_API_KEY` and `LATTICE_CHAT_API_KEY` were unset.

7.3 Verification gates

Required checks are `npm run test`, `node --check` for changed JavaScript, `python3 -m py_compile` for changed Python, `git diff --check`, SVG/XML parsing, PDF metadata/text checks, zero unresolved manuscript placeholders, and a LaTeX log scan for fatal errors and overfull boxes.

7.4 Render

The manuscript is rendered through the template repository's `scripts/pipeline/stage_03_render.py` against the working project. Renderer output is disposable; the tracked manuscript artifact is `docs/manuscript/artifacts/psw-vibelandia-sing13-manuscript-followup.pdf`.

7.5 Security and evidence boundary

Credentials are runtime inputs only. The deterministic receipt explicitly says it contains no provider calls or provider results. The raw live response text remains outside committed follow-up artifacts.

8 Document map and modularization

8.1 Manuscript modules

Module	Purpose
00_abstract.md	Abstract, contributions, honesty boundary
01_introduction.md	Background, research questions, architecture
02_formalism.md	Treatment model, outcomes, paired statistics, threats
03_methods.md	Task battery, protocol, runs, figures, system boundary
04_results.md	Primary, cross-model, per-task, and failure results
05_discussion.md	Mechanisms, applicability, limitations, recommendations
06_reproducibility.md	Artifact registry, commands, rendering, security
07_document_map.md	This map and document organization contract
99_references.md	Bibliography and repository references
config.yaml	Renderer metadata and output format configuration
preamble.md	LaTeX packages and layout additions
SYNTAX.md	Local Markdown/Pandoc syntax and label registry

8.2 Documents directory organization

The legacy `docs/` corpus contains many historically named whitepapers and technical notes. To preserve links and historical filenames, this pass does not rename or move legacy files. Instead, organization is provided through stable index documents and purpose-based collections:

- `docs/README.md` — top-level navigation and honesty boundary;
- `docs/INDEX_BY_TOPIC.md` — topic map for architecture, protocols, research, product/UI, and operations;
- `docs/INDEX_BY_LIFECYCLE.md` — seed/origin, implementation, evidence, and archive map;
- `docs/OPENROUTER_LATTICE_EXPERIMENT.md` — operational experiment protocol;
- `docs/manuscript/` — renderable research manuscript source;
- `reports/` — generated HTML/SVG experiment reports;
- `data/` — generated JSON/CSV evidence and historical comparison receipts.

Generated outputs are not copied into source sections. The manuscript points to report artifacts and reproduces key figures from the same aggregated data.

8.3 Seed:Edge document contract

Every document should identify:

1. **Seed:** the originating question, protocol, or formal claim;
2. **Edge:** the implementation, user experience, or empirical observation;
3. **Boundary:** what the document does not establish;
4. **Evidence:** source files, runs, tests, or citations;
5. **Lifecycle:** current, exploratory, superseded, or archival status.

This contract keeps conceptual whitepapers, implementation notes, and empirical reports from being silently treated as equivalent evidence.

8.4 Proposed future physical taxonomy

A future migration may place files into these directories after a link audit:

```
docs/
  architecture/    system and protocol architecture
  protocols/       operational protocol specifications
  research/        hypotheses, methods, and empirical reports
  product/         UI, catalog, and user-facing behavior
  operations/      deployment, access, and maintenance
  archive/         superseded or historical documents
```

This pass intentionally leaves the physical legacy corpus stable. The index-first approach is reversible and avoids breaking external references while the repository is actively evolving.

8.5 Rendering contract

The renderer consumes ordered numeric Markdown modules. Filenames are part of the composition protocol: lower numbers establish context and definitions; **99_references.md** terminates the body with references. Figures use relative paths under **docs/manuscript/figures/** and are generated before rendering.

9 References

This manuscript is primarily a software and empirical artifact report. External claims should be added here only when they are used in the prose and can be verified independently.

9.1 Repository sources

- `lib/openrouter-experiment.mjs` — experiment core, treatment constructors, scoring, paired statistics, and SVG/HTML report functions.
- `scripts/lattice-vs-standard-openrouter-usage.mjs` — live OpenRouter runner and result serializer.
- `lib/lattice-prompt.mjs` — Lattice prompt assembly and topology normalization.
- `lib/lattice-engine.mjs` — token accounting and structural execution envelopes.
- `tests/scripts/openrouter-experiment.test.mjs` — experiment-core tests.
- `tests/lib/lattice-prompt.test.mjs` — plain/direct topology tests.
- `docs/OPENROUTER_LATTICE_EXPERIMENT.md` — operational protocol note.

9.2 Run artifacts

- Primary three-treatment run: `data/openrouter-lattice-experiment-2026-08-08T04-34-20-127Z.json`.
- Primary visual report: `reports/openrouter-lattice-experiment-2026-08-08T04-34-20-127Z.html`.
- Valid cross-model run: `data/openrouter-lattice-experiment-2026-08-08T04-37-05-973Z.json`.
- Structural comparison receipt: `data/lattice-vs-standard-comparison.json`.

9.3 Citation policy

The current manuscript deliberately avoids unverifiable external citations. When related-work citations are added, use Pandoc citation syntax and add complete entries to `references.bib`; do not add citations merely to decorate the background.